Handling of Untrusted Data in the SpoilFree Extension, TVmaze Client and Model Loading

Divyam Saluja

GitHub: salujadivyam | LinkedIn: salujadivyam

Abstract, SpoilFree handles data from four untrusted sources: text on web pages, replies from its own API, replies from TVmaze and titles typed by the user. A review checked each path for script injection, path injection, unsafe parsing and unsafe model files. This report lists the findings and the measures in place. No exploitation was observed.

Index Terms, input validation, cross-site scripting, path traversal, deserialisation, safetensors.

I. INTRODUCTION

Data that crosses a trust boundary must be validated before it is used in markup, in a URL, in a regular expression or in a file loader. The relevant weakness classes are cross-site scripting [1], path traversal [2] and deserialisation of untrusted data [3].

II. REVIEW FINDINGS AND MEASURES

A. Markup. The content script and popup never assign untrusted strings to innerHTML. Page text is read with innerText and the popup writes show names with textContent, so a title that contains markup is shown as text [4]. Blur is applied by adding a class to the existing element and the tooltip is set as a property.

B. Regular expressions. Match terms are normalised to letters, digits and spaces before the pattern is built, so no pattern metacharacters reach the engine.

C. API replies. The content script accepts a reply only if its confidence is a number between 0 and 1. The background worker rejects messages that are not strings of at most 2000 characters and sends requests without credentials.

D. TVmaze replies. The fetcher now requires the show id to be an integer before it is placed in a URL path, checks that replies are arrays, requires names to be strings and caps their length, treats non-OK statuses as failures and aborts requests after five seconds with an abort controller [5].

E. User titles. The title field is limited to 80 characters and is encoded before it is used in the search URL.

F. Model files. The training notebook loads models with the safetensors option and the service sets remote code execution off when loading. Pickle-based model files can execute code when loaded, so safetensors is the safer format [3], [6].

III. VERIFICATION

A Node test replaced fetch with a fake. A show id of “1/../x” produced no cast or episode request, a null character name was skipped and a 429 reply to the episode call produced an empty list without an exception. The syntax of all changed files was checked. A browser-level injection test was not run.

IV. CONCLUSION

The design avoids the dangerous sinks instead of relying on filtering. Pinning dependency versions and checking the model file type before deployment remain open items.

REFERENCES

[1] MITRE, “CWE-79: Improper Neutralization of Input During Web Page Generation (‘Cross-site Scripting’).” Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/79.html

[2] MITRE, “CWE-22: Improper Limitation of a Pathname to a Restricted Directory (‘Path Traversal’).” Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/22.html

[3] MITRE, “CWE-502: Deserialization of Untrusted Data.” Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/502.html

[4] Mozilla, “Node: textContent property,” MDN Web Docs. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

[5] Mozilla, “AbortController,” MDN Web Docs. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/AbortController

[6] Hugging Face, “Safetensors.” Accessed: Oct. 28, 2026. [Online]. Available: https://huggingface.co/docs/safetensors/index